# Project 12: Onboarding Agent Tools Implementation & Testing Report

**Module:** Onboarding Agent Tools
**Assigned Roll Number:** 25071A0569
**Git Branch:** `feat/onboarding-agent-0569`
**Endpoint:** `POST /api/onboarding-agent`
**Repository:** open-source-contributor-project
**Status:** Completed, Integrated, Tested (33/33 Tests Passing), and Pushed to GitHub

## 1. Executive Summary

This document details the engineering work completed for the **Onboarding Agent Tools** module (assigned to roll number **25071A0569**). The module delivers an intelligent, advisory, and strictly read-only onboarding assistant designed to help new open-source contributors find suitable starter issues, inspect required prerequisites and documentation, and verify issue availability in real time.

The system was developed strictly according to the team conventions, utilizing Node.js 20+, Express.js, MongoDB, Mongoose, LangChain.js, and Zod.

## 2. Architecture & Workflow

## 3. What Was Implemented

### A. Three Read-Only Agent Tools ( `server/src/services/agent/` )

1. `findIssues` ( `findIssuesTool` )
   - **Location:** `server/src/services/agent/findIssues.js`
   - **Purpose:** Searches MongoDB for real open issues matching contributor goals, skills, and project criteria.

- **Key Implementation Details:**
  - Queries `status: 'open'` and `ownerId: null` (or `$exists: false`) exclusively.
  - Excludes assigned and closed issues from candidate results.
  - Supports optional `projectId`, `skills` (ranks results by skill match), and keyword matching from `goal` on `title` and `summary`.
  - Enforces result bounding (`limit` capped at 20) to prevent expensive queries.
  - Exposes both the direct function and a LangChain tool definition (`tool()` from `@langchain/core/tools`).

**2. `inspectPrerequisites` (`inspectPrerequisitesTool`)**

- **Location:** `server/src/services/agent/inspectPrerequisites.js`
- **Purpose:** Identifies everything a contributor needs before starting work on a specific issue.
- **Key Implementation Details:**
  - Validates `issueId` as a 24-character hexadecimal ObjectId.
  - Queries the real `Issue` record along with associated `Document` records (`setup`, `contributing`, `readme`) and `Snapshot` versioning.
  - Detects `staleSetupFlag: true` and surfaces appropriate maintainer warnings.
  - Flags `missingSetupInfo: true` when documentation is absent, without fabricating commands.
  - Returns structured citations satisfying `citationSchema`.

**3. `checkIssueAvailability` (`checkIssueAvailabilityTool`)**

- **Location:** `server/src/services/agent/checkIssueAvailability.js`
- **Purpose:** Verifies that an issue is actively open and unassigned.
- **Key Implementation Details:**
  - Inspects the latest record in MongoDB.
  - Marks issues unavailable if `status === 'assigned'`, `status === 'closed'`, or `ownerId` is populated.
  - Strictly read-only: does not assign, reserve, or alter issue status.

## B. Agent Coordinator Workflow (`server/src/services/agent/agentService.js`)

- **Function:** `runOnboardingAgent({ userId, projectId, goal, skills, hoursAvailable, limit })`
- **Demonstration Scenario:** "*Find a task I can start this weekend*"

- **Workflow Highlights:**
    1. Validates authenticated user identity.
    2. Inserts an `AgentRun` document with `status: 'running'`.
    3. Discovers candidate issues using `findIssues`.
    4. Sequentially screens each candidate:
        - Checks availability via `checkIssueAvailability`.
        - Rejects assigned or closed candidates and sets `revisedFrom` to the rejected issue ID.
        - Inspects prerequisites via `inspectPrerequisites`.
        - Re-checks availability immediately before recommendation to prevent race conditions.
    5. Selects the first fully available candidate and constructs a recommendation satisfying `agentRecommendationSchema`.
    6. Finalizes `AgentRun` with `status: 'completed'` and logs all executed tools with timing and summaries.

### C. Agent-Run Logging (`AgentRun` Model)

Reused the existing Mongoose model in `server/src/models/AgentRun.js`:

- Each tool call is tracked in `agentRun.toolCalls` with:
    - `name`: Name of the tool executed.
    - `args`: Sanitized input arguments (credentials/secrets redacted).
    - `status`: Allowed enum value (`'ok'`, `'error'`, or `'skipped'`).
    - `resultSummary`: Concise summary of the result.
    - `startedAt`: Timestamp when execution started.
    - `durationMs`: Execution duration in milliseconds.
- Captures failure states without corrupting workflow execution or returning internal stack traces.

### D. Authentication Middleware (`server/src/middleware/auth.js`)

- Created `requireAuth` to protect the agent endpoint.
- Validates active sessions from cookies (`sessionToken` or `session`) or `Authorization: Bearer <token>` against the `Session` model using SHA-256 token hashing.
- Attaches the verified user document to `req.user` and `req.userId`.

- Strictly prevents impersonation: client body-supplied `userId` values are ignored and rejected.

### E. API Route (`server/src/routes/agent.routes.js`)

- Endpoint: `POST /api/onboarding-agent`
- Auto-mounted under `/api` by `server/src/routes/index.js`.
- Validates the request body with Zod (`agentRequestSchema`).
- Invokes `runOnboardingAgent` using the authenticated `req.user._id`.

## 4. Changes Made to Original Content

A primary goal was **non-destructive integration**: preserving all existing features, schemas, constants, and tests.

### A. Modified File: `oss-onboarding/server/src/schemas/ai.js`

In the original repository skeleton, `agentRequestSchema` required `projectId: objectId` as mandatory, and `toolArgSchemas.findIssues` required both `{ skills, projectId }` strictly:

```
javascript
// ORIGINAL SKELETON
export const agentRequestSchema = z
  .object({
    projectId: objectId, // Required
    goal: z.string().trim().min(3).max(300).default('Find a task I can start this
weekend'),
    skills: skillList.optional(),
    hoursAvailable: z.number().min(1).max(80).optional(),
  })
  .strict();

export const toolArgSchemas = {
  findIssues: z.object({ skills: skillList, projectId: objectId }).strict(), // Both
required
  inspectPrerequisites: z.object({ issueId: objectId }).strict(),
  checkIssueAvailability: z.object({ issueId: objectId }).strict(),
};
```

**Why it was changed:**

1. Contributors querying for a starter task across all open-source projects should not be blocked if they have not yet chosen a specific `projectId`.
2. `findIssues` needs to support filtering by `goal` keyword hints and `limit` in addition to optional `skills` and `projectId`.

**Exact Modification Made:**

```
javascript
// UPDATED IMPLEMENTATION
export const agentRequestSchema = z
  .object({
    projectId: objectId.optional(), // Made optional
    goal: z.string().trim().min(3).max(300).default('Find a task I can start this
weekend'),
    skills: skillList.optional(),
    hoursAvailable: z.number().min(1).max(80).optional(),
  })
  .strict();

export const toolArgSchemas = {
  findIssues: z
    .object({
      skills: skillList.optional(),    // Made optional
      projectId: objectId.optional(), // Made optional
      goal: z.string().trim().max(500).optional(),
      limit: z.number().int().min(1).max(50).optional(),
    })
    .strict(),
  inspectPrerequisites: z.object({ issueId: objectId }).strict(),
  checkIssueAvailability: z.object({ issueId: objectId }).strict(),
};
```

**Backward Compatibility Verification:**

- In `tests/schemas.test.js`:
  - `agentRequestSchema.parse({ projectId: oid })` continues to parse with `goal: 'Find a task I can start this weekend'`.
  - `toolArgSchemas.checkIssueAvailability.safeParse({ issueId: oid })` remains 100% untouched.
  - All original 12 tests pass with zero regressions.

### B. Helper Configuration Files Added to Project Root & Server

1. `.gitignore` **(Repository Root):**
    - The initial repository lacked a `.gitignore`, causing `node_modules` and local artifacts to appear as untracked files.
    - Added standard ignore patterns for `node_modules/`, `.env`, and log files.
2. `server/.env.example`**:**
    - Provided an environment variable template with non-sensitive defaults for `MONGODB_URI`, `SESSION_SECRET`, and `CLIENT_ORIGIN`.

### C. Existing Files Kept Completely Intact (Unmodified)

- `server/src/app.js`
- `server/src/constants.js`
- `server/src/models/Issue.js`
- `server/src/models/AgentRun.js`
- `server/src/models/Project.js`
- `server/src/models/Document.js`
- `server/src/models/Snapshot.js`
- `server/src/models/Session.js`
- `server/src/models/User.js`
- `server/src/models/index.js`
- `server/src/routes/index.js`
- `server/src/routes/health.routes.js`
- `server/tests/app.test.js`
- `server/tests/models.test.js`
- `server/tests/schemas.test.js`

## 5. How to Test

### A. Automated Test Suite (Built-in Node.js Test Runner)

All unit, service, and API integration tests use isolated in-memory mocks. **A live MongoDB Atlas instance or running Ollama instance is NOT required.**

### 1. Running All Tests

Navigate to the `oss-onboarding/server` directory and execute:

```
bash
cd oss-onboarding/server
npm test
```

*For PowerShell environments, you can explicitly set test environment variables:*

```
powershell
$env:MONGODB_URI='mongodb://127.0.0.1:27017/test'; $env:SESSION_SECRET='test-secret-test-secret'; npm test
```

*For Linux / macOS environments:*

```
bash
MONGODB_URI='mongodb://127.0.0.1:27017/test' SESSION_SECRET='test-secret-test-secret' npm test
```

**Test Suite Breakdown (33 Tests Total):**

1. `tests/agentTools.test.js` **(12 tests):**
    - Rejection of invalid Zod inputs.
    - Discovery of open and unassigned issues; exclusion of assigned/closed issues.
    - Safe empty return when no issues exist.
    - LangChain tool wrapper execution.
    - Prerequisite and documentation inspection.
    - Stale setup warning flagging.
    - Safe handling of invalid and missing issue IDs.
    - Availability checking for open/unassigned issues.
    - Rejection of assigned issues, closed issues, and issues with an active `ownerId`.
    - Handling of missing/invalid records.
2. `tests/agentService.test.js` **(4 tests):**
    - End-to-end demonstration scenario ("*Find a task I can start this weekend*").
    - Rejection of an unavailable first candidate and selection of an eligible second candidate (`revisedFrom` tracking).
    - Guarantee that unavailable issues are never recommended when all candidates are taken.

- Transition of `AgentRun.status` to `'failed'` on unexpected database errors.

3. **`tests/agentApi.test.js` (5 tests):**
   - Rejection of unauthenticated requests (`401 Unauthorized`).
   - Rejection of expired/invalid session tokens (`401 Unauthorized`).
   - Rejection of invalid request bodies (`422 validation_error`).
   - Enforcement of authenticated user identity (ignoring spoofed body `userId`).
   - Shielding internal database errors from API clients (`500 Internal Server Error` without stack traces).
4. **Existing Baseline Tests (12 tests):**
   - `tests/app.test.js`: Health check (`200 OK`) and 404 handler (`2 tests`).
   - `tests/models.test.js`: Validation for User, Issue, Plan, Chunk, and Session schemas (`4 tests`).
   - `tests/schemas.test.js`: Zod schema validation for auth, plans, review, issue update, chat, agent request, and tool arguments (`6 tests`).

## B. Running Test Files Individually

You can execute each test file in isolation:

```
bash
# Run only Agent Tools tests
node --test tests/agentTools.test.js

# Run only Agent Workflow & Logging tests
node --test tests/agentService.test.js

# Run only API & Auth integration tests
node --test tests/agentApi.test.js

# Run original schema tests
node --test tests/schemas.test.js
```

## C. Manual / Live API Verification via cURL

When running the Express server (`npm run dev` or `node src/index.js`):

### 1. Verify Authentication Enforcement (Expected: 401 Unauthorized)

```
bash
curl -i -X POST http://localhost:5000/api/onboarding-agent \
  -H "Content-Type: application/json" \
  -d '{"goal": "Find a task I can start this weekend"}'
```

*Expected response code:* `401 Unauthorized`

### 2. Verify Zod Body Validation (Expected: 422 Unprocessable Entity)

```
bash
curl -i -X POST http://localhost:5000/api/onboarding-agent \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <valid-session-token>" \
  -d '{"goal": "ab"}'
```

*Expected response code:* `422 Unprocessable Entity` (goal string must have at least 3 characters).

### 3. Successful Agent Run (Expected: 200 OK)

```
bash
curl -i -X POST http://localhost:5000/api/onboarding-agent \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <valid-session-token>" \
  -d '{
    "goal": "Find a task I can start this weekend",
    "skills": ["javascript", "markdown"]
  }'
```

*Expected response code:* `200 OK` with `runId`, `status: "completed"`, `recommendation`, and full `toolCalls` execution logs.

## 6. Git Details & Verification

- **Branch:** `feat/onboarding-agent-0569`
- **Commit Hash:** `5c153e98d7a6410ac6fe267d0c8f7a0ed1c0b57c`
- **Commit Message:** `feat: implement onboarding agent tools`
- **Remote Push Status:** Succeeded ( `origin/feat/onboarding-agent-0569` )

- **Working Tree:** Clean (verified with `git status`)